

**INSTITUTO
FEDERAL**

Santa Catarina

Câmpus
São José

Cálculo Numérico

**Apostila com todos os métodos
numéricos**

2026-07-21

Hiago Ritter Santos (github.com/hiagosantos60)

Yan Vitor Pires Pereira (github.com/yanvpp)

IFSC

Sumário

I. Introdução	4
II. Método da bisseção	5
II.1. Definição e conceito	5
III. Método do ponto fixo	8
III.1. Definição e conceito	8
III.2. Passo a passo para verificação do método	8
IV. Método de Newton-Raphson (com derivadas)	10
IV.1. Definição e conceito	10
IV.2. Passo 1: Definir a função e sua derivada	10
IV.3. Passo 2: Escolher o chute inicial	10
IV.4. Passo 3: Aplicar a interatividade e atualizar os valores com base no erro e na tolerância	10
IV.5. Implementação	10
V. Método da secante (sem derivadas)	11
V.1. Definição e conceito	11
V.2. Passo a passo	11
V.3. Implementação	11
VI. Método de Eliminação de Gauss	12
VI.1. Definição e conceito	12
VI.2. Formulação	12
VI.3. Exemplo Completo com Pivoteamento	12
1. Matriz Inicial	12
2. 1º Pivoteamento (Coluna 1)	12
3. Eliminação na Coluna 1	13
4. 2º Pivoteamento (Coluna 2)	13
5. Eliminação na Coluna 2	13
6. Substituição Regressiva	13
VI.4. Exemplo de implementação	14
VII. Método de Newton (Vetorial)	15
VII.1. Definição e conceito	15
VII.2. Formulação	15
VII.3. Exemplo	15
VII.4. Implementação	16
VIII. Método de Lagrange	17
VIII.1. Definição e conceito	17
VIII.2. Construção dos polinômios auxiliares (3 pontos)	17
VIII.3. Exemplo	17
VIII.4. Implementação	18
IX. Método de Jacobi-Richardson	19
IX.1. Definição e conceito	19
Condições para poder usar o método:	19
IX.2. EXEMPLO COMPLETO	19
1ª Iteração	20

2ª Iteração	20
3ª Iteração	21
IX.3. Implementação	21
X. Método dos Mínimos Quadrados	22
X.1. Definição e conceito	22
X.2. Passo a passo	22
X.3. Exemplo: aproximação por parábola	22
X.4. Implementação para uma parábola a partir dos pontos	24
XI. Série de Taylor	25
XI.1. Definição e conceito	25
XI.2. Expansão de $\sin(x)$ em $x_0 = 0$	25
XI.3. Expansão de $\cos(x)$ em $x_0 = 0$	25
XI.4. Expansão de $\cos(x)$ em $x_0 = \pi$	25
XI.5. Aplicação: integral de e^{-x^2}	25
XII. Integração pelo Método do Trapézio	26
XII.1. Definição e conceito	26
XII.2. Implementação	26
XIII. Integração pelo Método de Simpson	27
XIII.1. Definição e conceito	27
XIII.2. Demonstração do padrão 1-4-1	27
XIII.3. Implementação	27
XIV. Diferenciação por Euler	28
XIV.1. Definição e conceito	28
Euler Básico	28
Euler Melhorado (Heun)	28
Euler Modificado (Ponto Médio)	29
Euler Atrasado (Implícito)	29
XIV.2. Diferenças entre as variantes	29
XV. Método de Runge-Kutta (4ª ordem)	31
XV.1. Definição e conceito	31
XV.2. Implementação	32

I. Introdução

Este material foi desenvolvido durante o semestre 2026.1 para a matéria de cálculo numérico pelos alunos Hiago Ritter Santos e Yan Vitor Pires Pereira. A matéria foi ministrada pelo professor Vitor Sales. A motivação para a criação do material é auxiliar novos alunos a compreenderem melhor o conteúdo no pós aula, pois requer algumas horas de estudo e dedicação. Esperamos que o conteúdo lhe ajude nessa jornada, não desista, foque no realmente importa e siga em frente!

II. Método da bisseção

II.1. Definição e conceito

o método da bisseção consiste em aproximar uma raiz de uma função a partir um determinado intervalo validado pelo teorema de Bolzano. O teorema de Bolzano consiste na seguinte regra: dado o intervalo $[a,b]$, se $f(a) < 0$ e $f(b) > 0$, sabe-se que naquele intervalo indicado haverá uma raiz para a função f . O contrário também é verdade, se $f(a) > 0$ e $f(b) < 0$, também pode-se afirmar que há uma raiz nesse intervalo, o que muda é se a função está subindo ou descendo naquele intervalo.

O intervalo pode ser tão pequeno quanto necessário, mas precisa respeitar o que o nosso teorema diz, caso o teorema não seja respeitado, nossa interação sobre o método não levará a lugar nenhum. Na implementação do código essa questão ficará evidente.

Vamos aplicar o método para a função $f(x) = x^2 + 2x - 2$

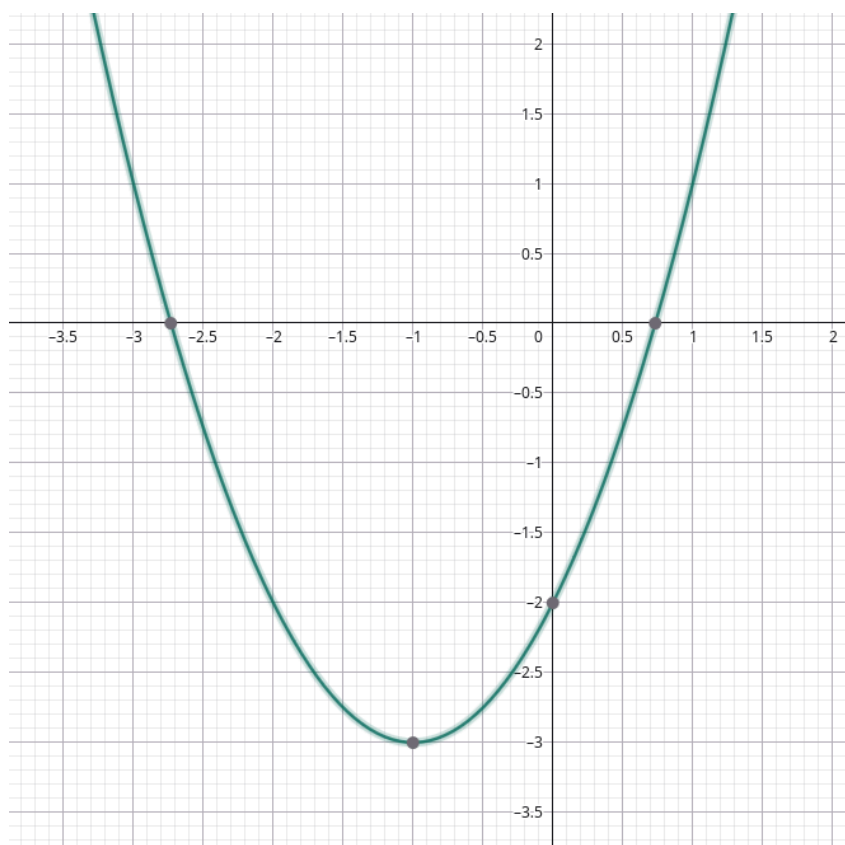


Figura 1 – Representação gráfica da função f

Observe que como temos uma função quadrática, vamos ter duas raízes em dois intervalos diferentes, onde podemos aplicar o Teorema de Bolzano e aplicar no código em si.

Para determinar os intervalos que contêm essas raízes, podemos analisar o comportamento da função avaliando alguns pontos estratégicos:

Avaliando o intervalo $[-3, -2]$:

$$\begin{aligned}f(-3) &= (-3)^2 + 2(-3) - 2 = 9 - 6 - 2 = 1 > 0 \\f(-2) &= (-2)^2 + 2(-2) - 2 = 4 - 4 - 2 = -2 < 0\end{aligned}\tag{1}$$

Avaliando intervalo $[0, 1]$:

$$\begin{aligned}f(0) &= (0)^2 + 2(0) - 2 = 0 + 0 - 2 = -2 < 0 \\f(1) &= (1)^2 + 2(1) - 2 = 1 + 2 - 2 = 1 > 0\end{aligned}\tag{2}$$

Agora que avaliamos e sabemos que a função tem raízes no intervalo $[0, 1]$ e no intervalo $[-3, -2]$, que chamaremos de nosso intervalo inicial $[a, b]$.

O primeiro passo é calcular o ponto médio (x_m) desse intervalo. Para o intervalo inicial de $[0, 1]$, fazemos:

$$x_m = \frac{a + b}{2} = \frac{0 + 1}{2} = 0.5\tag{3}$$

Para o intervalo $[-3, -2]$, o raciocínio é o mesmo, resultando em $x_m = -2.5$.

Depois de descobrir o ponto médio, deve-se comparar o seu resultado na função para verificar se a aproximação está no lado negativo ou positivo. Vamos avaliar o ponto médio $x_m = 0.5$:

$$f(0.5) = (0.5)^2 + 2(0.5) - 2 = -0.75 \quad (\text{resultado negativo})\tag{4}$$

É aqui que atualizamos o nosso intervalo $[a, b]$ como mostra o código. Lembre-se de que a raiz sempre estará onde ocorre a troca de sinal. Vamos comparar os extremos:

- No ponto $a = 0$, o resultado era **negativo** ($f(0) = -2$).
- No nosso ponto médio $x_m = 0.5$, o resultado também é **negativo** ($f(0.5) = -0.75$).
- No ponto $b = 1$, o resultado é **positivo** ($f(1) = 1$).

Como não há troca de sinal entre 0 e 0.5 (ambos são negativos), a raiz não está ali. A troca de sinal ocorre entre o 0.5 e o 1. Portanto, o nosso limite inferior a avança, e o novo intervalo atualizado passa a ser $[0.5, 1]$.

Se notar, há um balanço entre o ponto médio que faz o intervalo variar e com isso se aplica o método. Quanto mais se aproximar o seu intervalo de raiz da função, melhor será sua aproximação.

Abaixo está uma implementação do código em forma de template. Sugerimos que você entenda a lógica do código e declare a função, assim você conseguirá chegar nas aproximações da raiz da função

```

1  clear all; close all; clc;
2  % intervalo
3  a = termoinicial;
4  b = termofinal;
5  f = @(x) inserirfuncao;
6
7  % definindo erro e tolerancia
8  erro = 1;
9  tolerancia = 1e-5;
10
11 while abs(erro > tolerancia)
12     meio = (a + b) / 2
13     if (f(meio) * f(a)) < 0)
14         b = meio;
15     else
16         a = meio;
17     endif
18     erro = abs(f(meio));
19 endwhile
20 disp(meio) % meio vai ser a aproximação da raiz

```

III. Método do ponto fixo

III.1. Definição e conceito

O método consiste em calcular de forma iterativa (repetitiva) para encontrar raízes de uma função.

Podemos observar que no código temos a implementação do processo iterativo para obtenção da raiz. Pois, temos a função que queremos encontrar a raiz, temos a limitação de até onde precisamos ser precisos e estamos constantemente atualizando a variável anterior, ou seja, conseguimos calcular a precisão da resposta até quanto queiramos.

III.2. Passo a passo para verificação do método

1. Verificar se a raiz está no intervalo $[a, b]$

- Calcular $f(a)$ e $f(b)$. Se $f(a) \cdot f(b) < 0$, sabemos que existe uma raiz no intervalo.

2. Transformar $f(x) = 0$ em $x = \varphi(x)$

- Isso é: encontrar o valor em que a curva toca o zero.
- φ é moldado a partir da função original, então deve-se encontrar formas de representar φ em função de x .
- Abaixo estão formas de representar a mesma função, mas em φ s diferentes:

$$x^2 + 2x - 2 = 0 \implies x = \sqrt{2 - 2x} \implies x' = \frac{2 - x^2}{2} \quad (5)$$

3. Derivar e verificar se os valores convergem ou divergem

- Calcular as derivadas da função φ montada anteriormente e testar os dois valores extremos do intervalo na derivada.
- Precisamos que a derivada de $\varphi(x)$ seja menor que 1 em ambos os casos para ter convergência: $|\varphi'(x)| < 1$.
- Se for necessário determinar um intervalo de convergência, o intervalo precisa funcionar nos φ s e o intervalo precisa ter uma raiz na função original. Caso o intervalo funcione nos φ s, mas não funcione na função original, é preciso testar outro intervalo. Se o intervalo tiver raiz na função original, mas não funcionar nos φ s, a φ encontrada é inválida.

4. Escolha um valor inicial e atualize o erro até que ele seja tão pequeno quanto necessário

- $|x_{\text{novo}} - x_{\text{antigo}}| < \text{erro}$


```

1  clear all; close all; clc;
2
3  % definir chute inicial
4  chute = 1;
5
6  % definir erro e tolerância
7  erro = 1;
8  tolerancia = 1e-5;
9
10 % definir função e phi
11 f = @(x) inserirfuncao;
12 phi = @(x) inserirfuncao;
13
14 while (erro > tolerancia)
15     % calcular o próximo chute
16     proxChute = phi(chute);
17     % recalcular o erro
18     erro = abs(chute - proxChute);
19     % atualizar o chute para a próxima iteração
20     chute = proxChute;
21 endwhile
22
23 disp(chute)

```

IV. Método de Newton-Raphson (com derivadas)

IV.1. Definição e conceito

O método de Newton-Raphson exige que a função $f(x)$ seja derivável, pois utiliza a reta tangente à curva em cada ponto para se aproximar da raiz de forma muito mais rápida que os métodos anteriores.

IV.2. Passo 1: Definir a função e sua derivada

1. Escrever a função original: $f(x) = x^2 + 2x - 2$
2. Definir a derivada: $f'(x) = 2x + 2$

IV.3. Passo 2: Escolher o chute inicial

Nessa fase precisamos decidir um chute que acreditamos estar próximo do intervalo onde imaginamos que está a raiz da função.

IV.4. Passo 3: Aplicar a interatividade e atualizar os valores com base no erro e na tolerância

A fórmula de recorrência do método é dada por:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} \quad (6)$$

A cada interação, comparamos o novo valor com o anterior e calculamos o erro, repetindo o processo até que o erro seja menor que a tolerância definida.

IV.5. Implmentação

```
1  % FUNÇÃO E DERIVADA
2  f = @(x) x^2 + 2*x - 2;
3  df = @(x) 2*x + 2;
4  % chute entre [0,1], também podemos testar com outro intervalo
5  x1 = 0.5;
6
7  tolerancia = 0.0001;
8  erro = 1;
9
10 while (erro > tolerancia)
11     x2 = x1 - f(x1)/df(x1);
12     erro = abs(x2 - x1);
13     x1 = x2;
14 end
15 display(x1);
```

V. Método da secante (sem derivadas)

V.1. Definição e conceito

O método da secante é utilizado quando não é conveniente ou possível calcular a derivada da função. Em vez da reta tangente, o método utiliza retas secantes (retas que ligam dois pontos da curva) para se aproximar da raiz.

Para a escolha dos chutes iniciais, pode-se usar o Teorema de Bolzano: se $f(a) \cdot f(b) < 0$, sabemos que há uma raiz naquele intervalo.

V.2. Passo a passo

1. Escolher dois chutes iniciais, x_0 e x_1 (podem ser obtidos a partir do Teorema de Bolzano).
2. Aplicar a fórmula da secante para obter uma nova aproximação:

$$x_{\text{novo}} = x_1 - f(x_1) \cdot \frac{x_1 - x_0}{f(x_1) - f(x_0)} \quad (7)$$

3. Atualizar os pontos: o antigo x_1 passa a ser o novo x_0 , e o x_{novo} calculado passa a ser o novo x_1 .
4. Repetir o processo, recalculando o erro a cada interação, até que ele seja menor que a tolerância definida.

V.3. Implementação

```
1  f = @(x) x^2 + 2*x - 2;
2  x0 = 0; % Primeiro chute
3  x1 = 1; % Segundo chute
4  tolerancia = 0.0001;
5  erro = 1;
6
7  while (erro > tolerancia)
8      % FÓRMULA DA SECANTE
9      x_novo = x1 - f(x1) * (x1 - x0) / (f(x1) - f(x0));
10     erro = abs(x_novo - x1);
11     x0 = x1; % O x1 antigo vira o novo x0
12     x1 = x_novo; % O resultado atual vira o novo x1
13 end
14
15 display(x1);
```

VI. Método de Eliminação de Gauss

VI.1. Definição e conceito

O método de Eliminação de Gauss transforma um sistema linear $Ax = b$ em um sistema triangular superior equivalente por meio de operações elementares de linha, e depois resolve esse sistema por substituição regressiva.

Para evitar divisão por zero e melhorar a estabilidade numérica, usa-se o **pivoteamento parcial**: antes de cada etapa de eliminação, a linha com o maior valor absoluto na coluna-pivô é trocada para a posição do pivô.

VI.2. Formulação

Para definir o multiplicador utilizamos a seguinte regra:

Em uma etapa j , o pivô é o elemento $a_{j,j}$ e temos o elemento da linha que queremos zerar $a_{i,j}$. O pivô deve ser o que tem o maior valor absoluto da coluna.

$$m_{ij} = \frac{a_{i,j}}{a_{j,j}} \quad (8)$$

A nova linha vai ser a linha atual menos o multiplicador vezes a linha do pivô. Exemplo de uma operação de linha completa:

$$L_i \leftarrow L_i - \left(\frac{a_{i,j}}{a_{j,j}} \right) L_j \quad (9)$$

—

VI.3. Exemplo Completo com Pivoteamento

Sistema original:

$$\begin{cases} 5x + y + z = 7 \\ 2x + 4y + z = 7 \\ x + y + 3z = 5 \end{cases} \quad (10)$$

1. Matriz Inicial

$$\left(\begin{array}{ccc|c} 5 & 1 & 1 & 7 \\ 2 & 4 & 1 & 7 \\ 1 & 1 & 3 & 5 \end{array} \right) \quad (11)$$

2. 1º Pivoteamento (Coluna 1)

O maior valor em módulo na coluna 1 é 5 (já em L_1). Não há troca de linhas. O pivô é 5.

$$\left(\begin{array}{ccc|c} [5] & 1 & 1 & 7 \\ 2 & 4 & 1 & 7 \\ 1 & 1 & 3 & 5 \end{array} \right) \quad (12)$$

3. Eliminação na Coluna 1

A) Linha 2: $m_{21} = \frac{2}{5} = 0.4$

$$L_2 \leftarrow L_2 - 0.4L_1 \quad (13)$$

$$\begin{cases} a_{21} : 2 - (0.4 \cdot 5) = 0 \\ a_{22} : 4 - (0.4 \cdot 1) = 3.6 \\ a_{23} : 1 - (0.4 \cdot 1) = 0.6 \\ b_2 : 7 - (0.4 \cdot 7) = 4.2 \end{cases} \quad (14)$$

B) Linha 3: $m_{31} = \frac{1}{5} = 0.2$

$$L_3 \leftarrow L_3 - 0.2L_1 \quad (15)$$

$$\begin{cases} a_{31} : 1 - (0.2 \cdot 5) = 0 \\ a_{32} : 1 - (0.2 \cdot 1) = 0.8 \\ a_{33} : 3 - (0.2 \cdot 1) = 2.8 \\ b_3 : 5 - (0.2 \cdot 7) = 3.6 \end{cases} \quad (16)$$

Matriz após a primeira eliminação:

$$\left(\begin{array}{ccc|c} 5 & 1 & 1 & 7 \\ 0 & 3.6 & 0.6 & 4.2 \\ 0 & 0.8 & 2.8 & 3.6 \end{array} \right) \quad (17)$$

4. 2º Pivoteamento (Coluna 2)

Comparando $|3.6|$ e $|0.8|$, o maior é 3.6 (já em L_2). Não há troca. O pivô é 3.6.

$$\left(\begin{array}{ccc|c} 5 & 1 & 1 & 7 \\ 0 & [3.6] & 0.6 & 4.2 \\ 0 & 0.8 & 2.8 & 3.6 \end{array} \right) \quad (18)$$

5. Eliminação na Coluna 2

Linha 3: $m_{32} = \frac{0.8}{3.6} = \frac{2}{9} \approx 0.2222$

$$L_3 \leftarrow L_3 - \frac{2}{9}L_2 \quad (19)$$

$$\begin{cases} a_{32} : 0.8 - (0.2222 \cdot 3.6) = 0 \\ a_{33} : 2.8 - (0.2222 \cdot 0.6) = 2.6667 \\ b_3 : 3.6 - (0.2222 \cdot 4.2) = 2.6667 \end{cases} \quad (20)$$

Matriz escalonada final:

$$\left(\begin{array}{ccc|c} 5 & 1 & 1 & 7 \\ 0 & 3.6 & 0.6 & 4.2 \\ 0 & 0 & 2.6667 & 2.6667 \end{array} \right) \quad (21)$$

6. Substituição Regressiva

i) Última equação:

$$2.6667z = 2.6667 \rightarrow z = 1 \quad (22)$$

ii) Segunda equação:

$$3.6y + 0.6z = 4.2$$

$$3.6y + 0.6(1) = 4.2 \quad (23)$$

$$3.6y = 3.6 \rightarrow y = 1$$

iii) Primeira equação:

$$5x + y + z = 7$$

$$5x + 1 + 1 = 7 \quad (24)$$

$$5x = 5 \rightarrow x = 1$$

$$\text{Solução final: } x = 1, \quad y = 1, \quad z = 1 \quad (25)$$

VI.4. Exemplo de implementação

```

1  % Metodo de Eliminacao de Gauss com pivoteamento parcial
2  A = [5 1 1; 2 4 1; 1 1 3];
3  B = [7; 7; 5];
4  M = [A B];      % matriz aumentada [A|B]
5  [l, c] = size(A);
6  for j = 1:c-1 % percorre colunas
7      % localiza a linha com o maior valor absoluto na coluna j
8      [~, idx] = max(abs(M(j:end, j)));
9      LinhaPermuta = j - 1 + idx;
10     % troca de linhas
11     aux = M(j, :);
12     M(j, :) = M(LinhaPermuta, :);
13     M(LinhaPermuta, :) = aux;
14     for i = j+1:l % percorre linhas abaixo do pivo
15         m_ij = M(i, j) / M(j, j);
16         M(i, :) = M(i, :) - m_ij * M(j, :);
17     end
18 end
19 % Normaliza a diagonal principal
20 for i = 1:l
21     M(i, :) = M(i, :) / M(i, i);
22 end
23 % Substituicao regressiva
24 for i = l-1:-1:1
25     s = 0;
26     for j = i+1:c
27         s = s + M(i, j) * M(j, c+1);
28     end
29     M(i, c+1) = M(i, c+1) - s;
30 end
31 solucao = M(:, c+1)

```

VII. Método de Newton (Vetorial)

VII.1. Definição e conceito

O método de Newton vetorial é uma extensão do método de Newton-Raphson para sistemas de equações **não lineares**. Em vez de utilizar uma reta tangente (como no caso escalar), utiliza-se um **plano tangente** definido pela **Matriz Jacobiana** J , que contém todas as derivadas parciais do sistema.

VII.2. Formulação

Dado um sistema $F(x) = 0$, a expansão de Taylor de primeira ordem em torno do ponto atual x_k é:

$$F(x_k) + J(x_k)\Delta x \approx 0 \quad (26)$$

Isolando o incremento Δx :

$$J(x_k)\Delta x = -F(x_k) \quad (27)$$

O ponto é atualizado pela fórmula:

$$x_{k+1} = x_k + \Delta x \quad (28)$$

Ou de forma equivalente:

$$x_{k+1} = x_k - J(x_k)^{-1}F(x_k) \quad (29)$$

VII.3. Exemplo

Dado o sistema:

$$F(x, y) = \begin{pmatrix} x^2 + xy^2 - 2 \\ xy - 3xy^3 + 4 \end{pmatrix} \quad (30)$$

A Jacobiana é a matriz das derivadas parciais de cada linha em relação a cada variável:

$$J(x, y) = \begin{pmatrix} \frac{\partial F_1}{\partial x} & \frac{\partial F_1}{\partial y} \\ \frac{\partial F_2}{\partial x} & \frac{\partial F_2}{\partial y} \end{pmatrix} = \begin{pmatrix} 2x + y^2 & 2xy \\ y - 3y^3 & x - 9xy^2 \end{pmatrix} \quad (31)$$

A cada iteração, resolve-se o sistema linear:

$$\begin{pmatrix} 2x + y^2 & 2xy \\ y - 3y^3 & x - 9xy^2 \end{pmatrix} \begin{pmatrix} \Delta x \\ \Delta y \end{pmatrix} = \begin{pmatrix} -(x^2 + xy^2 - 2) \\ -(xy - 3xy^3 + 4) \end{pmatrix} \quad (32)$$

e atualiza-se o chute:

$$\begin{pmatrix} x_{\text{novo}} \\ y_{\text{novo}} \end{pmatrix} = \begin{pmatrix} x_{\text{antigo}} \\ y_{\text{antigo}} \end{pmatrix} + \begin{pmatrix} \Delta x \\ \Delta y \end{pmatrix} \quad (33)$$

VII.4. Implementação

```
1 % Método de Newton Vetorial
2 clear all; close all; clc;
3 % Chute inicial
4 chute = [3; 4];
5 % Critério de parada
6 erro = 1;
7 tolerancia = 1e-5;
8
9 while erro > tolerancia
10     x = chute(1);
11     y = chute(2);
12     % Vetor função F(x,y)
13
14     F = [ funcao1;
15           funcao2 ];
16     % Jacobiana J = [dF1/dx, dF1/dy; dF2/dx, dF2/dy]
17     J = [ d1x, d1y;
18           d2x, d2y ];
19
20     % Resolve J * dX = -F (operador \ usa eliminação de Gauss)
21     dX = J \ (-F);
22
23     % Critério de parada: norma do incremento
24     erro = norm(dX);
25
26     % Atualiza o chute
27     chute = chute + dX;
28 end
29 disp(chute)
```


VIII. Método de Lagrange

VIII.1. Definição e conceito

O método de Lagrange serve para encontrar o polinômio que passa exatamente por um conjunto de pontos conhecidos $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$. O polinômio interpolador é construído como uma soma ponderada de polinômios auxiliares $L_{i(x)}$:

$$P(x) = y_1 \cdot L_1(x) + y_2 \cdot L_2(x) + \dots + y_n \cdot L_n(x)$$

Cada polinômio auxiliar $L_{i(x)}$ é construído para valer **1** no ponto x_i e **0** em todos os outros pontos:

$$L_{i(x)} = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$$

VIII.2. Construção dos polinômios auxiliares (3 pontos)

Para três pontos $(x_1, y_1), (x_2, y_2), (x_3, y_3)$:

$$L_1(x) = \frac{(x-x_2)(x-x_3)}{(x_1-x_2)(x_1-x_3)}$$

$$L_2(x) = \frac{(x-x_1)(x-x_3)}{(x_2-x_1)(x_2-x_3)}$$

$$L_3(x) = \frac{(x-x_1)(x-x_2)}{(x_3-x_1)(x_3-x_2)}$$

O numerador zera nos pontos indesejados; o denominador normaliza o resultado para que $L_{i(x_i)} = 1$.

VIII.3. Exemplo

Dados os pontos $(1, 2), (2, 3), (4, 1)$:

$$P(x) = 2 \cdot \frac{(x-2)(x-4)}{(1-2)(1-4)} + 3 \cdot \frac{(x-1)(x-4)}{(2-1)(2-4)} + 1 \cdot \frac{(x-1)(x-2)}{(4-1)(4-2)}$$

$$P(x) = \frac{2(x-2)(x-4)}{3} - \frac{3}{2}(x-1)(x-4) + \frac{1}{6}(x-1)(x-2)$$

Com o polinômio montado, basta substituir qualquer x desejado para obter o valor interpolado com precisão matemática.

Quanto mais pontos forem conhecidos, mais termos o somatório terá. Para 10 pontos, por exemplo, L_1 ficaria:

$$L_1(x) = \frac{(x-x_2)(x-x_3)\dots(x-x_{10})}{(x_1-x_2)(x_1-x_3)\dots(x_1-x_{10})}$$

VIII.4. Implementação

```
1  X = [1, 2, 4];
2  Y = [2, 3, 1];
3  x_alvo = 3;
4  n = length(X);
5  Px = 0;
6
7  for i = 1:n
8      Li = 1;
9      for j = 1:n
10         % Não pode dividir por zero
11         if i ~= j
12             %  $Li = Li * [(x - x_j) / (x_i - x_j)]$ 
13             numerador = x_alvo - X(j);
14             denominador = X(i) - X(j);
15             Li = Li * (numerador / denominador);
16         end
17     end
18     Px = Px + Y(i) * Li;
19 end
20 fprintf('Resultado da interpolação dinâmica para x = %0.1f: %.4f\n', x_alvo, Px);
```

IX. Método de Jacobi-Richardson

IX.1. Definição e conceito

Se trata de um método **iterativo**, ou seja, requer múltiplas iterações para ser executado com sucesso.

Condições para poder usar o método:

1. NENHUM elemento da diagonal principal da matriz A pode ser 0.
2. **Critério das linhas:** Em todas as linhas, o módulo do elemento da diagonal principal precisa ser maior que a soma dos módulos de todos os outros elementos daquela mesma linha.
3. **Critério das colunas:** Em todas as colunas, o módulo do elemento da diagonal principal precisa ser maior que a soma dos módulos de todos os outros elementos daquela mesma coluna.

Serve para sistemas lineares $Ax = B$ de ordem n , por exemplo:

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n = b_2 \\ \vdots \\ a_{n,1}x_1 + a_{n,2}x_2 + \dots + a_{n,n}x_n = b_n \end{cases} \quad (34)$$

Podemos decompor a matriz em: $A = L + D + R$

$$A = \begin{pmatrix} 0 & \dots & 0 \\ a_{21} & \dots & 0 \\ \vdots & \vdots & \vdots \\ a_{n,1} & a_{n,2} & 0 \end{pmatrix} + \begin{pmatrix} a_{11} & \dots & 0 \\ 0 & \dots & 0 \\ \vdots & \vdots & \vdots \\ 0 & \dots & a_{nn} \end{pmatrix} + \begin{pmatrix} 0 & a_{12} & a_{1n} \\ 0 & \dots & 0 \\ \vdots & \vdots & \vdots \\ 0 & \dots & 0 \end{pmatrix} \quad (35)$$

O próximo passo seria normalizar nossas matrizes dividindo toda a linha pelo elemento que pertence à diagonal. O que irá gerar $A^* = L^* + I + R^*$.

Após isso, devemos isolar o próximo valor, gerando o método iterativo:

$$x^{(k+1)} = b^* - (L^* + R^*)x^{(k)} \quad (36)$$

O método depende que você defina um chute inicial e, após cada iteração com o método, ele irá atualizar esse valor que definimos inicialmente. Com isso, conseguimos descobrir o valor de cada variável. Nota-se após algumas iterações que temos uma convergência acontecendo para os números.

IX.2. EXEMPLO COMPLETO

Sistema original:

$$\begin{cases} 10x_1 + 2x_2 + x_3 = 7 \\ x_1 + 5x_2 + x_3 = -8 \\ 2x_1 + 3x_2 + 10x_3 = 6 \end{cases} \quad (37)$$

Forma matricial:

$$\begin{pmatrix} 10 & 2 & 1 \\ 1 & 5 & 1 \\ 2 & 3 & 10 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 7 \\ -8 \\ 6 \end{pmatrix} \quad (38)$$

Decomposição:

$$A = \begin{pmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ 2 & 3 & 0 \end{pmatrix} + \begin{pmatrix} 10 & 0 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & 10 \end{pmatrix} + \begin{pmatrix} 0 & 2 & 1 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{pmatrix} \quad (39)$$

Processo de normalização das linhas:

$$\begin{aligned} L_1 : \frac{10}{10}x_1 + \frac{2}{10}x_2 + \frac{1}{10}x_3 &= \frac{7}{10} \Rightarrow x_1 + 0.2x_2 + 0.1x_3 = 0.7 \\ L_2 : \frac{1}{5}x_1 + \frac{5}{5}x_2 + \frac{1}{5}x_3 &= -\frac{8}{5} \Rightarrow 0.2x_1 + x_2 + 0.2x_3 = -1.6 \\ L_3 : \frac{2}{10}x_1 + \frac{3}{10}x_2 + \frac{10}{10}x_3 &= \frac{6}{10} \Rightarrow 0.2x_1 + 0.3x_2 + x_3 = 0.6 \end{aligned} \quad (40)$$

Sistema isolando x_1, x_2, x_3 depois de normalizar:

$$\begin{cases} x_1^{(k+1)} = 0.7 - 0.2x_2^{(k)} - 0.1x_3^{(k)} \\ x_2^{(k+1)} = -1.6 - 0.2x_1^{(k)} - 0.2x_3^{(k)} \\ x_3^{(k+1)} = 0.6 - 0.2x_1^{(k)} - 0.3x_2^{(k)} \end{cases} \quad (41)$$

Os k e $k + 1$ não são expoentes, são apenas para indicar a iteração que foi feita.

Agora com 3 iterações no método ficaria assim:

1ª Iteração

$$\begin{aligned} x_1^{(1)} &= 0.7 - 0.2(0) - 0.1(0) = 0.7 \\ x_2^{(1)} &= -1.6 - 0.2(0) - 0.2(0) = -1.6 \\ x_3^{(1)} &= 0.6 - 0.2(0) - 0.3(0) = 0.6 \\ \mathbf{x}^{(1)} &= [0.7, \quad -1.6, \quad 0.6]^T \end{aligned} \quad (42)$$

2ª Iteração

$$\begin{aligned} x_1^{(2)} &= 0.7 - 0.2(-1.6) - 0.1(0.6) = 0.96 \\ x_2^{(2)} &= -1.6 - 0.2(0.7) - 0.2(0.6) = -1.86 \\ x_3^{(2)} &= 0.6 - 0.2(0.7) - 0.3(-1.6) = 0.94 \\ \mathbf{x}^{(2)} &= [0.96, \quad -1.86, \quad 0.94]^T \end{aligned} \quad (43)$$

3ª Iteração

$$\begin{aligned}x_1^{(3)} &= 0.7 - 0.2(-1.86) - 0.1(0.94) = 0.978 \\x_2^{(3)} &= -1.6 - 0.2(0.96) - 0.2(0.94) = -1.98 \\x_3^{(3)} &= 0.6 - 0.2(0.96) - 0.3(-1.86) = 0.966 \\x^{(3)} &= [0.978, \quad -1.98, \quad 0.966]^T\end{aligned}\tag{44}$$

Podemos notar a convergência dos valores na tabela abaixo:

k	$x_1^{(k)}$	$x_2^{(k)}$	$x_3^{(k)}$
0	0	0	0
1	0.7	-1.6	0.6
2	0.96	-1.86	0.94
3	0.978	-1.98	0.966
...	↓	↓	↓
∞	1.0	-2.0	1.0

IX.3. Implementação

```
1  a=[10 2 1; 1 5 1; 2 3 10]; % 3x3
2  b=[7; -8; 6]; % 3x1
3  %%DIVISÃO DAS MATRIZES PELO ELEMENTO DA DIAGONAL PRINCIPAL
4  [m n]=size(a);
5  for i=1:m
6      divisor = a(i,i);
7      b(i,:) = b(i,+)/divisor;
8      a(i,:) = a(i,+)/divisor;
9      a(i,i) = 0;
10 end
11 %chute inicial
12 x0 = [0;0;0]; % antigo 3x1
13 x1 = b-a*x0; % novo 3x1
14 erro = max(abs(x1-x0));
15 while(erro > 0.01)
16     x1 = b-a*x0;
17     % Forma verbosa de fazer a atualização
18     % for i=1:size(a)
19     %     x1(i) = b(i) - a(i,:)*x0;
20     % end
21     erro = max(abs(x1-x0));
22     x0 = x1; % atualiza o proximo chute inicial
23 end
24 x1
25 x0
```

X. Método dos Mínimos Quadrados

X.1. Definição e conceito

O método dos mínimos quadrados aproxima uma função que passe o mais **próximo possível** de um conjunto de pontos, minimizando a soma dos quadrados dos erros entre os valores reais y_i e os valores previstos pelo modelo.

X.2. Passo a passo

1. Definir a função modelo, por exemplo:

- Reta: $f(x) = ax + b$
- Parábola: $f(x) = ax^2 + bx + c$
- Exponencial: $f(x) = ae^x + b$

2. Definir a equação do erro:

$$E = \sum_i [(ax_i + b) - y_i]^2 \quad (45)$$

3. Minimizar o erro derivando parcialmente em relação a cada parâmetro e igualando a zero:

$$\frac{\partial E}{\partial a} = \sum 2(ax_i + b - y_i) \cdot x_i = 0 \quad (46)$$

$$\frac{\partial E}{\partial b} = \sum 2(ax_i + b - y_i) \cdot 1 = 0 \quad (47)$$

4. Organizar o sistema linear resultante e resolver.

Desenvolvendo as derivadas para uma reta, chega-se ao sistema:

$$\begin{cases} a \sum x_i^2 + b \sum x_i = \sum x_i y_i \\ a \sum x_i + bn = \sum y_i \end{cases} \quad (48)$$

X.3. Exemplo: aproximação por parábola

Para o modelo $f(x) = ax^2 + bx + c$:

O erro total E (resíduo quadrático) é definido pela soma dos quadrados dos desvios entre o valor real y_i e o modelo estimado:

$$E = \sum [(ax_i^2 + bx_i + c) - y_i]^2 \quad (49)$$

- Minimizando os erros (Derivadas Parciais):

Para encontrar os coeficientes que minimizam o erro, igualamos as derivadas parciais a zero utilizando a regra da cadeia:

$$\frac{\partial E}{\partial a} = \sum 2(ax_i^2 + bx_i + c - y_i) \cdot x_i^2 = 0 \quad (50)$$

$$\frac{\partial E}{\partial b} = \sum 2(ax_i^2 + bx_i + c - y_i) \cdot x_i = 0 \quad (51)$$

$$\frac{\partial E}{\partial c} = \sum 2(ax_i^2 + bx_i + c - y_i) \cdot 1 = 0 \quad (52)$$

- **Desenvolvimento e distribuição dos termos:**

Dividindo cada equação por 2 e separando os somatórios com as incógnitas a , b e c do lado esquerdo, isolamos os termos com y_i no lado direito:

1. Da derivada em relação a a :

$$a \sum x_i^4 + b \sum x_i^3 + c \sum x_i^2 = \sum x_i^2 y_i \quad (53)$$

2. Da derivada em relação a b :

$$a \sum x_i^3 + b \sum x_i^2 + c \sum x_i = \sum x_i y_i \quad (54)$$

3. Da derivada em relação a c (lembrando que $\sum c = cn$):

$$a \sum x_i^2 + b \sum x_i + cn = \sum y_i \quad (55)$$

- **Montando o sistema com as equações encontradas:**

$$\begin{cases} a \sum x_i^4 + b \sum x_i^3 + c \sum x_i^2 = \sum x_i^2 y_i \\ a \sum x_i^3 + b \sum x_i^2 + c \sum x_i = \sum x_i y_i \\ a \sum x_i^2 + b \sum x_i + cn = \sum y_i \end{cases} \quad (56)$$

X.4. Implementação para uma parábola a partir dos pontos

```
1  x = [-0.95, -0.65, -0.35, -0.05, 0.25, 0.55, 0.85, 1.15, 1.45];
2  y = [-2.57, -2.01, -1.61, -1.42, -1.33, -1.33, -1.48, -1.85, -2.38];
3
4  n = length(x);
5
6  A = [sum(x.^4), sum(x.^3), sum(x.^2);
7       sum(x.^3), sum(x.^2), sum(x);
8       sum(x.^2), sum(x),    n];
9
10 B = [sum(x.^2 .* y); sum(x .* y); sum(y)];
11
12 K = A \ B;
13 a = K(1); b = K(2); c = K(3);
14
15 dominio = min(x):0.01:max(x);
16 imagem = a*(dominio.^2) + b*dominio + c;
17 plot(x, y, '*', dominio, imagem, '-');
18 legend('Pontos', 'Parábola ajustada');
```


XI. Série de Taylor

XI.1. Definição e conceito

A Série de Taylor aproxima funções complexas em torno de um ponto x_0 por meio de um somatório de derivadas:

$$f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(x_0)(x-x_0)^n}{n!} \quad (57)$$

XI.2. Expansão de $\sin(x)$ em $x_0 = 0$

As derivadas de $\sin(x)$ avaliadas em $x_0 = 0$ seguem o padrão $0, 1, 0, -1, 0, 1, \dots$. Apenas os termos ímpares sobrevivem:

$$\sin(x) = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n+1}}{(2n+1)!} = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots \quad (58)$$

XI.3. Expansão de $\cos(x)$ em $x_0 = 0$

As derivadas de $\cos(x)$ avaliadas em $x_0 = 0$ seguem o padrão $1, 0, -1, 0, 1, \dots$. Apenas os termos pares sobrevivem:

$$\cos(x) = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n}}{(2n)!} = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \dots \quad (59)$$

XI.4. Expansão de $\cos(x)$ em $x_0 = \pi$

As derivadas avaliadas em π produzem a sequência $-1, 0, 1, 0, -1, \dots$, com sinal $(-1)^{n+1}$ nos termos pares:

$$\cos(x)_{x_0=\pi} = \sum_{n=0}^{\infty} \frac{(-1)^{n+1} (x-\pi)^{2n}}{(2n)!} = -1 + \frac{(x-\pi)^2}{2!} - \frac{(x-\pi)^4}{4!} + \dots \quad (60)$$

XI.5. Aplicação: integral de e^{-x^2}

Como $\int e^{-x^2} dx$ não possui forma fechada elementar, expande-se a exponencial em série e integra-se termo a termo:

$$\int_{-1}^1 e^{-x^2} dx = \int_{-1}^1 \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n}}{n!} dx = 2 \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n+1)n!} \quad (61)$$

XII. Integração pelo Método do Trapézio

XII.1. Definição e conceito

O método do trapézio aproxima a integral de $f(x)$ no intervalo $[a, b]$ por trapézios formados entre pontos consecutivos da função. Com apenas dois pontos (um trapézio):

$$\int_a^b f(x)dx \approx (b-a) \frac{f(a) + f(b)}{2} \quad (62)$$

Para n subintervalos de tamanho $h = \frac{b-a}{n}$ (trapézios múltiplos):

$$\int_a^b f(x)dx \approx \frac{h}{2} \left[f(x_0) + 2 \sum_{i=1}^{n-1} f(x_i) + f(x_n) \right] \quad (63)$$

Os extremos $f(x_0)$ e $f(x_n)$ entram uma vez; todos os pontos internos entram com peso 2.

XII.2. Implementação

```
1  clear; close all; clc;
2
3  f = @(x) cos(x);
4  a = 0;
5  b = pi/2;
6  n = 10;      % número de trapézios
7
8  h = (b - a) / n;
9  ponto = a:h:b;
10
11 soma_internos = 0;
12 for i = 2:n
13     soma_internos = soma_internos + f(ponto(i));
14 end
15
16 area = (h/2) * (f(a) + 2*soma_internos + f(b));
17 fprintf('Área aproximada: %f\n', area);
```

XIII. Integração pelo Método de Simpson

XIII.1. Definição e conceito

O método de Simpson ajusta parábolas a grupos de três pontos igualmente espaçados. O número de subintervalos n **deve ser par**. A fórmula composta é:

$$\int_a^b f(x)dx \approx \frac{h}{3} \left[f(x_0) + 4 \sum_{k=1}^{\frac{n}{2}} f(x_{2k-1}) + 2 \sum_{k=1}^{\frac{n}{2}-1} f(x_{2k}) + f(x_n) \right] \quad (64)$$

Os pontos de índice **ímpar** (centrais de cada parábola) recebem peso 4; os de índice **par** internos recebem peso 2.

XIII.2. Demonstração do padrão 1-4-1

Integrando a parábola genérica $f(x) = Ax^2 + Bx + C$ no intervalo $[-h, h]$:

$$\int_{-h}^h (Ax^2 + Bx + C)dx = \frac{2Ah^3}{3} + 2Ch = \frac{h}{3}(2Ah^2 + 6C) \quad (65)$$

Avaliando nos três pontos:

- $f(-h) + f(h) = 2Ah^2 + 2C$
- $f(-h) + 4f(0) + f(h) = 2Ah^2 + 6C$

Portanto:

$$\int_{x_0}^{x_2} f(x)dx = \frac{h}{3}[f(x_0) + 4f(x_1) + f(x_2)] \quad (66)$$

XIII.3. Implementação

```
1  f = @(x) x.^3;
2  a = 0;
3  b = 2;
4  n = 100; % deve ser par
5  h = (b - a) / n;
6  ponto = a:h:b;
7
8  soma_impares = sum(f(ponto(2:2:end-1))); % índices ímpares → peso 4
9  soma_pares = sum(f(ponto(3:2:end-2))); % índices pares internos → peso 2
10 area = (h/3) * (f(ponto(1)) + 4*soma_impares + 2*soma_pares + f(ponto(end)));
11
12 fprintf('Resultado da integral: %f\n', area);
```

XIV. Diferenciação por Euler

XIV.1. Definição e conceito

O método de Euler resolve numericamente **Equações Diferenciais Ordinárias** (EDOs) quando não é possível encontrar a solução analítica exata. A ideia parte da definição de derivada por limite:

$$y' = \lim_{\Delta x \rightarrow 0} \frac{y(x + \Delta x) - y(x)}{\Delta x} \quad (67)$$

Usando um passo finito h , isolamos o próximo valor:

$$y_{i+1} = y_i + h \cdot f(x_i, y_i) \quad (68)$$

Euler Básico

O método avança usando a inclinação **no início** de cada passo.

```
1 f = @(x, y) -2*y;  
2 h = 0.1;  
3 x = 0:h:2;  
4 y = zeros(size(x));  
5 y(1) = 1;  
6  
7 for i = 2:length(x)  
8     k1 = f(x(i-1), y(i-1));  
9     y(i) = y(i-1) + h * k1;  
10 end  
11  
12 plot(x, y, 'o-', x, exp(-2*x), 'r-');  
13 legend('Euler Básico', 'Solução Exata');
```

Euler Melhorado (Heun)

Calcula a inclinação no início (k_1) e no fim estimado (k_2) do passo, e usa a média:

$$k_1 = f(x_n, y_n) \quad (69)$$

$$k_2 = f(x_n + h, y_n + h \cdot k_1) \quad (70)$$

$$y_{n+1} = y_n + \frac{h}{2}(k_1 + k_2) \quad (71)$$

```

1  f = @(x, y) y;
2  h = 0.1;
3  x = 0:h:1;
4  y = zeros(size(x));
5  y(1) = 1;
6
7  for i = 2:length(x)
8      k1 = f(x(i-1), y(i-1));
9      k2 = f(x(i-1) + h, y(i-1) + h*k1);
10     y(i) = y(i-1) + (h/2) * (k1 + k2);
11 end
12
13 disp(y(end));

```

Euler Modificado (Ponto Médio)

Estima o ponto médio do passo com k_1 e usa a inclinação nesse ponto central para avançar:

$$k_1 = f(x_n, y_n) \quad (72)$$

$$k_2 = f\left(x_n + \frac{h}{2}, y_n + \left(\frac{h}{2}\right) \cdot k_1\right) \quad (73)$$

$$y_{n+1} = y_n + h \cdot k_2 \quad (74)$$

Euler Atrasado (Implícito)

Não existe um código genérico: é necessário discretizar e isolar y_i analiticamente para cada EDO específica. Para $f(x, y) = -2y$:

$$y_i = \frac{y_{i-1}}{1 + 2h} \quad (75)$$

Para cada EDO será necessário fazer a discretização manualmente e encontrar o que será melhor para sua EDO

XIV.2. Diferenças entre as variantes

O **Euler Básico** usa apenas a inclinação calculada no início do intervalo (k_1) para avançar o passo inteiro. Isso o torna simples e rápido, mas o erro acumulado cresce mais rapidamente, já que a inclinação real muda ao longo do passo e não é corrigida.

O **Euler Melhorado (Heun)** tenta compensar essa limitação calculando uma segunda inclinação (k_2) na extremidade final do passo — estimada a partir do próprio k_1 e usa a média entre k_1 e k_2 . Isso captura melhor a curvatura da função e reduz o erro em relação ao método básico, ao custo de uma avaliação extra de f .

O **Euler Modificado (Ponto Médio)** segue uma lógica parecida, mas em vez de estimar a inclinação no fim do passo, estima a inclinação no **meio** do intervalo, usando $\frac{h}{2}$ para projetar esse ponto médio. A inclinação central tende a representar melhor o comportamento da função no passo inteiro do que a inclinação inicial isolada.

Já o **Euler Atrasado (Implícito)** se diferencia dos demais porque usa a inclinação avaliada no **próprio ponto futuro** y_i , e não em y_{i-1} . Isso o torna mais estável para EDOs rígidas, mas exige isolar y_i algebricamente a cada equação, já que ele aparece nos dois lados da expressão e por isso não existe uma implementação genérica em loop como nos outros três métodos.

XV. Método de Runge-Kutta (4ª ordem)

XV.1. Definição e conceito

O método de Runge-Kutta de 4ª ordem é o mais eficiente entre os métodos estudados em termos de convergência para a solução real. Ele utiliza quatro “sondas” de inclinação ao longo do passo e combina-as em uma média ponderada:

$$k_1 = f(x_n, y_n) \quad (76)$$

$$k_2 = f\left(x_n + \frac{h}{2}, y_n + \left(\frac{h}{2}\right)k_1\right) \quad (77)$$

$$k_3 = f\left(x_n + \frac{h}{2}, y_n + \left(\frac{h}{2}\right)k_2\right) \quad (78)$$

$$k_4 = f(x_n + h, y_n + hk_3) \quad (79)$$

$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4) \quad (80)$$

k_1 usa a inclinação no início; k_2 e k_3 refinam a estimativa no ponto médio; k_4 verifica o final. Os pesos 1, 2, 2, 1 refletem a importância maior dos pontos centrais.

XV.2. Implementação

Implementação de código para exemplo da EDO $y' = y$, que tem solução analítica e^x .

```
1  clear all; close all; clc;
2  f = @(x, y) y; % EDO: substitua pela sua função
3  h = 0.5;
4  x = 0:h:1;
5  y = zeros(size(x));
6  y(1) = 1; % condição inicial
7  for i = 2:length(x)
8      xn = x(i-1);
9      yn = y(i-1);
10     k1 = f(xn, yn);
11     k2 = f(xn + h/2, yn + (h/2)*k1);
12     k3 = f(xn + h/2, yn + (h/2)*k2);
13     k4 = f(xn + h, yn + h*k3);
14     y(i) = yn + (h/6)*(k1 + 2*k2 + 2*k3 + k4);
15 end
16 fprintf('Valor final: %.6f\n', y(end));
```

matlab

Código 1 – Runge-Kutta de 4ª ordem

1	<code>clear all; close all; clc;</code>	matlab
2	<code>f = @(x, y) y;</code>	
3	<code>ERRO = zeros(1, 10);</code>	
4	<code>Hs = zeros(1, 10);</code>	
5	<code>for j = 1:10</code>	
6	<code>h = 0.5 / (2^j);</code>	
7	<code>x = 0:h:1;</code>	
8	<code>y = zeros(size(x));</code>	
9	<code>y(1) = 1;</code>	
10	<code>for i = 2:length(x)</code>	
11	<code>k1 = f(x(i-1), y(i-1));</code>	
12	<code>k2 = f(x(i-1) + h/2, y(i-1) + (h/2)*k1);</code>	
13	<code>k3 = f(x(i-1) + h/2, y(i-1) + (h/2)*k2);</code>	
14	<code>k4 = f(x(i-1) + h, y(i-1) + h*k3);</code>	
15	<code>y(i) = y(i-1) + (h/6)*(k1 + 2*k2 + 2*k3 + k4);</code>	
16	<code>end</code>	
17	<code>ERRO(j) = abs(exp(1) - y(end));</code>	
18	<code>Hs(j) = h;</code>	
19	<code>end</code>	
20	<code>LogHs = log(Hs);</code>	
21	<code>LogERRO = log(ERRO);</code>	
22	<code>coef = (LogERRO(end) - LogERRO(1)) / (LogHs(end) - LogHs(1));</code>	
23	<code>fprintf('Ordem de convergência: %.4f\n', coef); % esperado ≈ 4</code>	
24	<code>plot(LogHs, LogERRO, '-o');</code>	
25	<code>xlabel('log(h)'); ylabel('log(Erro)');</code>	
26	<code>title('Convergência — Runge-Kutta 4ª ordem');</code>	
27	<code>grid on;</code>	

Código 2 – Runge-Kutta — análise de convergência